ChatGPT

Project Opportunities in Computational Biology for the Nebius × NVIDIA Hackathon

Executive Summary: This report surveys past NVIDIA/Nebius hackathon projects and state-of-the-art AI in computational biology, then proposes 4–6 hackathon project ideas for a bioengineering/CS student. We first summarize 8–12 representative projects (primarily from NVIDIA and Nebius events) in computational biology, highlighting their problems, methods, models, data, GPU resources, and success factors. Next, we review recent advances (last 5 years) in AI for computational biology – e.g. protein folding, molecule generation, genomics, single-cell, imaging – citing key papers and tools. We then propose several candidate hackathon projects (problem, approach, data, Nvidia stack, compute, metrics, demo) and rank them by novelty, impact, and feasibility. Finally, we detail architectures for the top two ideas with Mermaid diagrams, optimization tips for NVIDIA GPUs, a 6–8 week plan, and enumerate open datasets/code, licensing, and ethics/regulatory considerations. A comparison table contrasts ideas by effort, novelty, impact, and resources. All sources are cited.

1. Survey of Past Hackathon Projects in Computational Biology

We reviewed NVIDIA- and Nebius-affiliated hackathons and challenges related to life sciences. Notably, the **CMU–NVIDIA Federated Learning Hackathon (Jan 2026)** produced ~10 teams' end-to-end prototypes on biomedical tasks, all leveraging **NVIDIA FLARE** (federated learning) and open biomed datasets【17†L308-L317】. Representative projects include:

- **FedGen (Synthetic GWAS data):** Generated realistic synthetic genomics data to mimic biobanks and trained a federated logistic regression for a binary trait【19†L329-L338】. (Models: logistic regression, federated updates; Data: synthetic GWAS; Compute: NVIDIA GPUs; Success: enabled realistic federated GWAS benchmarking.)
- **FedPathHarmony (Histopathology harmonization):** Addressed domain shift in multi-site pathology slides by applying stain normalization and style transfer within FLARE, so federated training avoids learning site-specific artifacts【19†L342-L350】. (Models: image-style-transfer networks; Data: CAMELYON histology slides; Compute: GPUs; Success: improved cross-site generalization.)
- **FedViz (Federated metadata visualization):** Built a web-based "command center" to visualize cohort and metadata differences across biobanks, helping teams audit data readiness for federated studies【19†L358-L366】. (Tools: GPU-accelerated dashboards; Data: harmonized open biobank metadata; Success: enabled pre-training exploration and improved study design.)
- **OmniGenome (Pangenome graphs federated):** Developed a FLARE framework to build pangenome graphs from distributed data: used 1000 Genomes variants to construct Alzheimer's-risk pangenome without exposing raw sequences【19†L369-L378】. (Models: distributed hashing and graph methods; Data: human pangenome reference, 1000 Genomes; Compute: GPUs; Success: showed federated pangenomes can match centralized graphs.)
- **Med_SNP_Deconvolution (Ancestry inference):** Produced an end-to-end pipeline that converts phased SNP data into encrypted "haploblock cluster" features, then trains ancestry classifiers via

FLARE【19†L391-L399】. (Models: GPU-accelerated ML over discrete cluster features; Data: genotype panels; Compute: GPUs; Success: enabled cross-site ancestry modeling without sharing raw genotypes.)

- **RAIDers (Rare-disease subtyping):** Simulated an ALS patient cohort by merging ClinVar variants and gnomAD frequencies to create >8000 synthetic patients across sites. Built a federated pipeline to discover genetic subtypes of ALS【19†L400-L409】. (Models: federated clustering; Data: ClinVar, gnomAD; Compute: GPUs; Success: proved multi-institution rare-disease analyses without centralizing data.)
- **OncoLearn (Federated multi-omic cancer subtyping):** Integrated RNA-seq and clinical data for breast cancer TCGA using federated transfer learning. Compared federated training vs. single-site transfer learning, achieving high subtype accuracy while respecting data borders【19†L419-L428】. (Models: federated neural nets; Data: TCGA multi-omics; Compute: GPUs; Success: high cross-site accuracy via federated transfer.)
- **PRSAggregator (Polygenic Risk Scores FL):** Enabled privacy-preserving federation of polygenic risk models by collaboratively calibrating and aligning PRS across cohorts, improving interpretability of PRS distributions globally【19†L441-L449】. (Models: federated linear models; Data: GWAS effect sizes; Compute: GPUs; Success: federated pipelines standardize PRS across ancestries.)
- **MuFFLe (Multi-modal FL):** Created a federated learning framework that fuses RNA-seq, clinical and imaging data for cancer prognosis. It handles modality gaps (e.g. a site missing one modality) by selectively disabling encoders, then training a collaborative model with encrypted updates【20†L471-L480】. On bladder cancer recurrence data, it stratified patients into distinct risk clusters with interpretable attention maps【20†L484-L489】. (Models: federated multi-modal neural net; Data: CHIMERA bladder cancer challenge; Compute: NVIDIA GPUs; Success: scalable federated multimodal prognosis.)
- **FedProFit (Protein fitness prediction):** Used **BioNeMo's ESM-2 (650M)** protein language model as a frozen backbone, and trained lightweight predictors on federated sites (viral, prokaryote, etc) for protein mutation fitness. Data were drawn from ~2.4M variants (ProteinGym). Demonstrated a single federated model predicting fitness across domains without revealing sequences【20†L500-L509】. (Models: large pLM + federated heads; Data: DMS assays (ProteinGym); Compute: NVIDIA GPUs; Success: cross-domain protein modeling via FLARE【20†L500-L509】.)

In addition, the **Nebius Serverless AI Builders Challenge (Aug 2026)** saw winners across healthcare and AI. The grand prize, **MediSimplifier**, fine-tuned 7–8B LLMs (using LoRA) to simplify clinical discharge summaries【22†L41-L49】. (Approach: fine-tune open LLMs on complex medical texts; Data: hospital discharge notes; Compute: H100 GPUs on Nebius token factory; Success: rewrote clinical text at patient-readable level.) Other awardees included medical-imaging examples (e.g. fMRI adversarial robustness【23†L109-L118】) and document AI (legal KYC), showing effective use of GPU cloud and serverless tools in HCLS.

These hackathon efforts generally used **NVIDIA GPUs (A100/H100)** on cloud (e.g. Nebius AI Cloud), NVIDIA's FLARE toolkit for federated training, and open biomedical datasets (TCGA, CAMELYON, LINCS, ClinVar, gnomAD, ProteinGym, etc) to push frontier problems. Their success came from combining domain knowledge with scalable AI and GPU compute, often addressing privacy or heterogeneity in biomedical data. We will draw on these lessons and tools for our proposed projects.

2. Recent Advances in AI for Computational Biology

Over the last 5 years, AI has revolutionized many areas of biology:

- **Protein Structure & Design:** DeepMind's *AlphaFold2* (2021) achieved near-experimental accuracy in protein folding, while RoseTTAFold (2021) and others similarly improved modeling. Building on protein language models (pLMs) like Meta's ESM-2 (2022) or Facebook's ProtTrans, frameworks like NVIDIA's **BioNeMo** enable training enormous models: e.g. BioNeMo v5 trained a 3B-parameter BERT-style pLM on >1 trillion protein tokens using 256 NVIDIA A100 GPUs in 4.2 days【40†L55-L60】. These pLMs, combined with diffusion models (e.g. "DiffDock" for docking【57†L0-L3】) are pushing generative design of proteins and small molecules. NVIDIA researchers also demonstrated *context-parallelism* to shard very large proteins across GPUs: e.g. folding a 3,605-residue complex in under 5 minutes on four H100s【54†L35-L38】.

- **Molecular Generation:** Generative models (RNNs, VAEs, GANs, graph neural networks) have long been applied to molecules. Recently, large-scale diffusion models and transformers ("ChemGPT") can generate realistic SMILES or conformers. NVIDIA's BioNeMo (also in 2026) supports on-demand molecular design pipelines. Emergent tools like LLMs for chemistry and enzyme design accelerate de novo drug discovery.

- **Genomics & Single-Cell:** AI models power variant calling (e.g. Google's DeepVariant) and GWAS. Deep-learning autoencoders (e.g. scVI, 2018【48†L0-L3】) now analyze single-cell RNA-seq at scale, handling dropouts and batch effects. Tools like scANVI, totalVI enable integrated multi-modal single-cell analysis. Machine-learning also aids interpretation of spatial transcriptomics and multimodal omics. Recent work uses graph neural nets for gene regulatory networks.

- **Imaging and Pathology:** Convolutional networks (U-Nets) have become standard for microscopy and histopathology segmentation. *Cellpose* (2021, Nature Biotech) offered a generalist cell segmenter across modalities, while newer segmentation models (e.g. transformer-based) continue to improve. NVIDIA's **Clara** toolkit supports medical image AI and DICOM workflows. AI has enabled state-of-the-art results in tumor detection (CAMELYON, BraTS) using GPUs and techniques like self-supervised learning.

- **Synthetic Biology:** AI assists in designing genetic circuits, promoter sequences, and optimizing metabolic pathways. For example, *DNABERT* (2022) applied transformers to genomic sequences, while generative models are now used to propose CRISPR guide RNAs or optimize codon usage. Efforts in metabolic flux prediction and enzyme engineering increasingly leverage ML.

- **Multi-Omics Integration:** Integrating genomics, transcriptomics, proteomics, etc., often requires embedding heterogeneous data. Deep models (autoencoders, graph embeddings) have improved biomarker discovery. Platforms like NVIDIA's RAPIDS accelerate large-scale data pre-processing (UMAP, PCA on GPU) for omics.

These advances – enabled by GPU compute and frameworks like CUDA/cuDNN – provide fertile ground for hackathon projects. We will leverage NVIDIA NeMo/Llama (for large bio-LLMs), Triton inference serving,

Clara imaging tools, RAPIDS for data, and Jetson/Edge if needed. Key recent references (papers, code) will be cited or listed for each specific technique as appropriate.

3. Proposed Project Ideas

We propose **6 project ideas**, each plausible for a 48–72h hackathon (or a multi-week extension). They are ranked roughly by novelty, potential impact, and feasibility (given Nebius/NVIDIA resources). Each idea includes problem, approach, data, Nvidia stack, resources, milestones, metrics, and demo.

1. **Federated Multi-Omic Cancer Prognosis (MuFFLe++).** *Problem:* Stratify cancer patients into risk subtypes using genomics + clinical data from multiple hospitals without sharing raw data.
   Approach: Building on the MuFFLe concept【20†L463-L472】, develop a federated learning pipeline where each "site" trains a local model on its data modalities (gene expression + EHR variables) and shares encrypted model updates via NVIDIA FLARE. Use a multi-modal neural net with separate encoders (e.g. TabNet/Transformer for omics, MLP for clinical) fused at aggregation. Handle missing modalities by gating encoders. Integrate this with Nebius Token Factory and Triton – model deployment as NVIDIA Triton Inference endpoints.
   Data: Use public multi-omic cancer cohorts (e.g. TCGA breast cancer: RNA-seq + clinical) and simulate multiple "sites" by partitioning data【19†L419-L428】【20†L463-L472】. Possibly incorporate imaging (digitized histology) as a bonus via Clara.
   NVIDIA Stack: **CUDA/cuDNN** for GPU training; **NVIDIA FLARE** for federation; **Triton** for model serving; **RAPIDS** for GPU-accelerated preprocessing (UMAP/clustering); **Jetson** if showing on-device inference prototype.
   Compute: ~2–4 H100 GPUs (federated clients + central); Nebius credit for H100s; possible use of Nebius Serverless AI jobs.
   Milestones: (a) Preprocess and split TCGA data; (b) Implement federated training loop (local training + global aggregation) with FLARE on Nebius; (c) Train/test and optimize model; (d) Deploy final model to Triton endpoint; (e) Evaluate Kaplan-Meier separation and F1 risk classification.
   Metrics: Prognosis accuracy (C-index) across cohorts; Generalization gap (federated vs centralized); Privacy (no raw data moved).
   Demo: Web dashboard showing federated training rounds, and an inference UI where a new patient's multi-omic profile yields subtype and survival curve.

2. **Protein Sequence-to-Function Predictor (FedProSpec).** *Problem:* Predict the effect of protein mutations (fitness/stability) across domains (human, viral, bacterial) without sharing sequences between labs.
   Approach: Similar to FedProFit【20†L491-L500】, use a pre-trained protein language model (e.g. NVIDIA BioNeMo pLM or Meta ESM-2) as a fixed backbone. Each client fine-tunes a small prediction head on its local mutation dataset (e.g. DeepSequence or ProteinGym subsets). Use **NVIDIA FLARE** to aggregate head updates. Provide an API via Triton for predictions on new mutations.
   Data: Use publicly available deep mutational scanning (DMS) datasets (ProteinGym: ~2.4M variants)【20†L500-L509】. Partition by protein family or organism.
   NVIDIA Stack: **NeMo Megatron** for model fine-tuning; **FLARE** for federated coordination; **Triton** for the final ensemble predictor; **CUDA** obviously; **RAPIDS** optional for data.
   Compute: 2–4 high-end GPUs (training frozen transformer is expensive; maybe leverage Nebius Token Factory to access Nemotron models).
   Milestones: (a) Fine-tune a protein pLM (650M–3B) on one dataset; (b) Implement federated loop for

other datasets; (c) Benchmark federated predictions vs centralized; (d) Optimize inference speed; (e) Setup Triton API.
Metrics: Prediction accuracy (Spearman correlation with experimental fitness); Performance (inference latency); Cross-domain improvement via FL.
Demo: Query interface: input a protein sequence and mutation → return predicted effect (e.g. stability change). Visualization of federated contribution (which sites benefited).

3. **Drug Repurposing via Gene Expression Profiles (RePurpoSense).** *Problem:* Identify approved drugs that could treat rare disease or cancer by matching disease gene expression signatures to drug perturbation profiles.
Approach: Leverage the LINCS L1000 dataset (drug vs cell-line expression) and disease signature datasets. Compute similarity between disease-induced gene expression changes and drug signatures using GPU-accelerated methods (e.g. RAPIDS cuML for correlation). Optionally train a neural model (e.g. Siamese network) to map expression to a shared embedding for drugs and diseases. Use Nebius GPUs to fine-tune a BiLSTM or Transformer on expression data.
Data: Public data from **Connectivity Map** (L1000), GEO (disease vs control expression), DrugBank for mappings. For novelty, target a specific case (e.g. lung cancer, or a rare neuro disease with GEO data).
NVIDIA Stack: **CUDA/cuDNN** for model; **RAPIDS cuML** for fast similarity search; **Triton** to serve the matching model or retrieval system; **NeMo** LLM for summarizing drug effects (optional agent).
Compute: 1–2 GPUs (for model and evaluation), Nebius credits.
Milestones: (a) Extract and preprocess expression signatures; (b) Build similarity pipeline (fast GPU dot-products / ML); (c) Train embedding model if doing learning; (d) Validate top repurposed drugs via literature; (e) Deploy as demo REST API.
Metrics: Enrichment of known drug-disease matches (precision/recall); Ranking speed.
Demo: Web interface: enter a disease or upload expression, outputs ranked list of candidate drugs with confidence scores and supporting evidence.

4. **Automated Histopathology Cancer Classification (DeepPathAI).** *Problem:* Classify tumor type or grade from whole-slide histology images (e.g. identify malignant vs benign regions).
Approach: Use a CNN/Transformer on histopathology tiles. Pretrain on ImageNet or use a pretrained model (e.g. BioReye or CLAM). Apply NVIDIA **Clara Deploy** or PyTorch for segmentation/ classification. Use Triton for serving the inference model. Possibly incorporate Nvidia Video Codec for large WSI I/O and RAPIDS for feature analysis. Optionally experiment with multi-agent (like Balagan) adversarial scenarios to test robustness.
Data: Public histopath datasets, e.g. CAMELYON16 (lymph node metastases), BRATS (brain tumor MRI/WSI), or Kaggle PCam (patch-level).
NVIDIA Stack: **Clara Train/Infer** (medical imaging toolkit); **CUDA/cuDNN**; **Triton Inference Server** (to host segmentation model); possibly **NeMo Megatron** if adding a pathology LLM for report generation (edge case).
Compute: 1 GPU for training (U-Net or ResNet); Triton can run on a cheap A100 or even L40S.
Milestones: (a) Prepare image patches and labels; (b) Train classifier/segmenter; (c) Evaluate performance (AUROC, F1); (d) Package model in Triton; (e) Build simple GUI: upload slide, overlay predictions.
Metrics: Classification accuracy on held-out slides; Speed (tiles/s on GPU); GPU utilization efficiency.
Demo: GUI or Jupyter widget showing slide image with tumor regions highlighted, powered by Triton on Nebius.

5. **GPU-Accelerated Single-Cell Analysis Platform (ScRAPIDS).** *Problem:* Perform end-to-end single-cell RNA-seq analysis (normalization, clustering, visualization) at GPU speed, and integrate with Nebius tools.
   Approach: Combine NVIDIA **RAPIDS** and **cuML** libraries for PCA/UMAP clustering with a small deep autoencoder. The key is to demonstrate orders-of-magnitude speedup on a large atlas (e.g. 1M cells). Possibly wrap into a Streamlit or Napari UI for exploration. Optionally fine-tune a single-cell variational autoencoder (scVI) model on GPU using PyTorch+CUDA. Use Triton to serve a pre-trained clustering model.
   Data: Public scRNA-seq atlases (e.g. Human Cell Atlas or Tabula Muris). Use open datasets of ~1M cells (maybe downsample to simulate scale).
   NVIDIA Stack: **RAPIDS cuDF/cuML** for data preprocessing (filtering, normalization), UMAP; **CUDA/cuDNN** for any deep model; **Triton** for serving precomputed embeddings.
   Compute: 1 GPU (data fits in memory); Nebius credits for throughput testing.
   Milestones: (a) Load and preprocess large scRNA data on GPU; (b) Implement GPU-accelerated PCA+UMAP; (c) (Optional) Train/use scVI on GPU; (d) Visualize clusters; (e) Deploy as interactive plot.
   Metrics: Runtime (vs CPU baseline); Clustering quality (ARI vs known labels); Memory efficiency.
   Demo: A web app showing 2D UMAP of cells colored by cluster or marker expression, interactively computed on the GPU.

6. **Biomedical Text Agent (BioQueryCopilot).** *Problem:* Build an intelligent assistant that reads biomedical papers or drug labels and answers queries.
   Approach: Use NVIDIA **NeMo Megatron** to fine-tune a biomedical LLM (e.g. BioNeMo LLM or open BioBERT) on scientific text (PubMed abstracts, drug monographs). Combine with **Triton** endpoints to serve the model. Optionally integrate with Nebius Token Factory's search API (Tavily) for retrieving references.
   Data: PubMed Central open-access corpus, UMLS/MeSH for terminology. Possibly S2ORC biomedical subset.
   NVIDIA Stack: **NeMo Megatron** for LLM fine-tuning; **CUDA/cuDNN**; **Triton** for hosting the chatbot model; **Jetson** could showcase an edge demo (e.g. voice interface).
   Compute: Several GPUs for fine-tuning (Nemotron); inference could run on a single H100 or L40 on Nebius.
   Milestones: (a) Prepare biomedical corpus; (b) Fine-tune or prompt-engineer an LLM; (c) Build a simple Q&A web/chat interface; (d) Evaluate factuality vs baseline GPT; (e) Add retrieval augmentation if time.
   Metrics: Answer accuracy on a biomedical QA benchmark; Latency per query; Model size/training cost.
   Demo: Chatbot interface where user types a question like "Which drug targets IL-6 receptor?" and the model answers with citations (via BLAS retrieval).

Each idea balances novelty and feasibility. For example, 1) and 2) target federated learning (novel, high-impact in privacy-preserving medicine) but require moderate effort to build FL pipelines. Idea 6 leverages recent LLM advances (novel) but may be heavy. Ideas 3–5 are more "classical" but still impactful and hack-friendly. We prioritize ideas that use NVIDIA stacks (Nemotron, Clara, RAPIDS, Triton) as requested.

4. Detailed Plan for Top 2 Ideas

4.1. Idea 1: Federated Multi-Omic Cancer Prognosis

Problem: Clinical cancer centers have different data (genomic profiles, clinical labs, possibly imaging) but cannot share raw patient data. We want a model that predicts patient risk subtypes (e.g. 3-year survival) by learning across centers without raw data exchange.

Technical Approach:

- Use **NVIDIA FLARE** to orchestrate federated training. Each "client" node trains on local data; only encrypted weight updates are shared with the Nebius-hosted FLARE server.
- Model: A multi-branch neural network. One branch (deep MLP or autoencoder) encodes RNA-seq counts; another branch encodes clinical features (age, stage, labs) via an MLP. Optionally, a third branch takes pre-extracted image features (ResNet) if pathology images are included. The branches are concatenated into a final classifier (risk score or subtype label). Weights are partially shared across sites.
- Use **CUDA/cuDNN** on H100s for model training.
- Serve the final model via **Triton Inference Server** on Nebius for demo.
- Use **RAPIDS** for GPU-accelerated preprocessing (e.g. normalization, PCA/UMAP for diagnostics).

```
flowchart LR
  subgraph Client 1 (Site A)
    A1[Patient Omics Data<br/>(RNA + Clinical)] --> B1{Local Trainer}
    B1 -- update --> C[Local Model]
  end
  subgraph Client 2 (Site B)
    A2[Patient Omics Data] --> B2{Local Trainer}
    B2 -- update --> C[Local Model]
  end
  subgraph Server
    D[Federation Server (FLARE on Nebius)]
  end
  C -->|send weights| D
  D -->|aggregate| C
  C -->|send updated| B1 & B2
```

Above, each client (site) trains its copy of the model on local H100 GPUs. The weights are periodically sent to the FLARE server on Nebius (using NVIDIA's secure aggregation). The server averages them (FedAvg) and returns global weights. This continues for many rounds.

Data Pipeline:

- **Dataset:** Use TCGA BRCA (breast cancer) cohort as proxy for one site, TCGA LUAD as another, etc. Each site's data is kept separate. We simulate 3–4 "sites" by splitting publicly available data by cohort/centers.

- **Preprocessing:** On Nebius GPUs (via RAPIDS), filter genes (e.g. top 10k variable genes), log-normalize counts, scale clinical features. Use PyTorch DataLoader with custom collate for federated batches.
- **Training:** Each site's FL client loads its partition. Use mixed-precision (Tensor Cores) on H100 to speed training. GPU profiling (Nsight Systems) ensures high utilization. Gradient checkpointing can allow larger batch.
- **Triton Deployment:** Once trained, package the global model in a Triton ensemble: GPU backend (TensorRT optimization of model) with pre/post-processing containers. Also deploy a small Flask UI that queries the Triton endpoint for inference.

Model Selection: Likely a simple fully-connected network or small transformer is sufficient. We can start with a 1D-CNN or MLP for gene expression (like scVI's encoder) plus MLP for clinical, then combine. We'll try both single-task (survival regression or classification) or multitask (subtype classification + survival). Key is to demonstrate FL works: compare the federated model's accuracy vs models trained on each site alone (local-only) and vs an impossible "centralized" model (for benchmark).

GPU Optimization:

- **Batching and FP16:** Use large batch sizes across tokens to amortize H100 startup. Enable PyTorch's `torch.cuda.amp` for half-precision.
- **CUDA Graphs:** If the model inference is small, consider CUDA Graphs to further reduce latency for Triton.
- **Memory:** Use NVLink efficiently; ensure each site can stream data asynchronously.
- **Profiling:** Use Nsight or PyTorch Profiler to identify bottlenecks. Likely I/O of large RNA matrices; we mitigate by pre-converting to RAPIDS cuDF and moving batches to GPU memory.

Timeline (6–8 weeks):
1. **Week 1:** Data preparation – gather TCGA data, split into 3 sites; set up Nebius environment. Implement preprocessing pipeline (RAPIDS). Preliminary PyTorch model training on one site to verify.
2. **Week 2:** Integrate NVIDIA FLARE – containerize the model for FLARE clients and server. Test local training + weight exchange.
3. **Week 3:** Train federated model (e.g. 50 rounds); tune learning rate, batch. Start evaluation (C-index, accuracy).
4. **Week 4:** Optimize model architecture (e.g. add dropout, try transfer learning for Omics). Implement alternative fusion strategies.
5. **Week 5:** Deployment – export model, set up Triton container. Develop simple UI (Streamlit or Flask).
6. **Week 6:** Testing and debugging – ensure end-to-end: data in → Triton inference → UI. Prepare demo script & slides.
7. **Weeks 7–8:** Write documentation, finalize code and blog, polish presentation. Run scalability tests (how many GPUs/hours).

4.2. Idea 2: Protein Sequence-to-Function Predictor

Problem: Predict functional fitness (e.g. enzyme activity) from sequence variants while respecting data privacy across labs (e.g. pharma, academia). This extends FedProFit【20†L491-L500】 to a new context.

Technical Approach:

- Use a large **protein language model** (e.g. NVIDIA's BioNeMo ESM-650M or ProtBERT) as a feature extractor. Freeze its weights.
- Each site (domain) fine-tunes a small downstream head (e.g. 2-layer MLP) on its mutation dataset.
- Or, if time permits, fine-tune entire model on federated data using NeMo Megatron (but that's heavy).
- Use **NVIDIA FLARE** for federated learning of the head.
- Serve final combined model via Triton.

```
flowchart LR
  subgraph Dataset Sites
    Seq1[ProteinSeq1 (Human)] --> Loc1{Local GPU} --> W1
    Seq2[ProteinSeq2 (Viral)] --> Loc2{Local GPU} --> W2
    Seq3[ProteinSeq3 (Bacterial)] --> Loc3{Local GPU} --> W3
  end
  subgraph FLServer
    FedServer((Fed Server on Nebius))
  end
  W1 -->|send weights| FedServer
  W2 -->|send weights| FedServer
  W3 -->|send weights| FedServer
  FedServer -->|aggregate| Wfed
  Wfed --> W1 & W2 & W3
  Wfed --> Triton[Deployment]
```

Data Pipeline:

- **Dataset:** Use open DMS datasets. For example, *ProteinGym* (substitution benchmarks)【20†L500-L509】 or *ProTherm*. Partition by protein family or lab origin to simulate privacy (e.g. one site has influenza HA mutants, another has bacterial enzyme mutants).
- **Preprocessing:** Use RAPIDS/cuDF to load sequences, encode one-hot or BLOSUM features. Use PyTorch DataLoader to batch.
- **Training:** On each site, use PyTorch on a Nebius GPU: pass sequences through frozen BioNeMo pLM (embedding output), then through local MLP head. Compute MSE or classification loss (depending on fitness label type). Use AdamW with small learning rate for head.
- **Federation:** After each epoch/round, send head weights to central server, average them (FedAvg), and broadcast updated weights. NVIDIA FLARE handles this securely【19†L329-L338】.
- **Serving:** The final model is the combination of pLM+averaged head. Export to ONNX or TorchScript and serve with Triton (possibly with TensorRT optimization).

Model Details: Pre-trained BioNeMo or ESM-2 embeddings (650M) as feature vectors. The head could be ~1M parameters (so cheap to train). We may experiment with adding attention to the head for capturing epistasis. Given privacy context, we follow FLARE guidelines: no raw sequences leave site.

GPU Optimization:

- **Mixed Precision:** The pLM inference is heavy; use FP16. Data parallel if model too large.
- **Model Pipeline Parallel:** If using Megatron, pipeline across GPUs (but likely skip due to time).
- **CUDA Graphs:** Possibly for inference across many mutations quickly.
- **Batching:** Large batches of sequences at once to feed GPU.

Timeline (6–8 weeks):
1. **Week 1:** Collect DMS data, simulate 3 sites. Test one-site baseline model training (embedding+MLP).
2. **Week 2:** Set up FLARE framework. Containerize head training. Run simple FL round with dummy data.
3. **Week 3:** Full FL training for multiple rounds, track loss convergence.
4. **Week 4:** Evaluate federated vs separate vs centralized models. Adjust network size/hyperparams.
5. **Week 5:** Deploy final model with Triton; create REST API for sequence input.
6. **Week 6:** Develop demo app or command-line tool to input sequence & get fitness. Prepare example case studies.
7. **Weeks 7–8:** Polish, write documentation, address edge cases.

5. Datasets, Codebases, Licensing, and Ethics

Open Datasets: We will leverage many public resources:

- **Cancer Omics:** TCGA (genomics, transcriptomics, clinical) via GDC; CHIMERA challenge for bladder cancer; TCIA for imaging.
- **Genomics/Variants:** 1000 Genomes, gnomAD, ClinVar【19†L400-L409】 for SNPs and rare disease variants; GTEx for expression.
- **Transcriptomics:** GEO datasets for specific diseases; LINCS L1000 (drug-induced signatures); ArrayExpress.
- **Proteins:** UniProt and ProteinGym (Proteome Workbench) for variant effect datasets; ProTherm.
- **Histology:** CAMELYON16 (lymph node metastasis); Kaggle PCam; TCGA images.
- **Single-Cell:** Human Cell Atlas, 10x Genomics PBMC public data.
- **Biomedical Text:** PubMed Central open articles, S2ORC, UMLS.

Codebases/Tools: We will reuse and adapt existing open-source projects when possible:

- **NVIDIA BioNeMo Framework** (GitHub)【40†L55-L60】 – for protein models and context-parallel folding.
- **NVIDIA FLARE** (GitHub) – for federated learning orchestration.
- **PyTorch/Lightning** – for model training.
- **RAPIDS cuML/cuDF** – for GPU data processing (Apache 2.0 license).
- **NVIDIA Clara Train/Deploy** – for medical image AI (Apache 2.0).
- **Hugging Face Transformers** – for any pre-trained LLM (Model license varies; many are Apache 2.0 or MIT).
- **Scikit-learn, scVI** – for baseline methods (BSD license).
  All chosen tools are open-source. We will respect licenses (MIT/Apache/etc). For example, LLMs like Llama 2 have specific terms (should check free non-commercial if needed).

Ethical and Regulatory Issues:

- **Privacy:** Many ideas (federated models) address patient data privacy by design (no raw data leaves hospital). We must emphasize and document that, aligning with HIPAA/GDPR concerns.
- **Bias:** Datasets (especially genetic or imaging) can be biased by ancestry or demographics. For example, PRS and genomics models often underperform on non-European ancestries. We will acknowledge any ancestry imbalances and, if possible, measure fairness metrics (e.g. model performance by subgroup) and discuss mitigation.
- **Misuse:** Our models (e.g. drug suggestion or protein design) should include disclaimers that they are research tools, not medical advice.
- **Data Licensing:** Ensure data used is open-access or properly licensed (TCGA/GEO are open; proprietary clinical notes are not used). Reiterating hackathon rules, no proprietary patient data is allowed.
- **Reproducibility:** All code will be public (MIT/Apache). We will containerize environments for reproducibility.

6. Comparison of Project Ideas

Idea	Effort (person-days)	Novelty	Potential Impact	GPU Resources
1. Federated Cancer Prognosis	High (≥6 wks)	High (privacy)	High (precision med)	High (H100×3+)
2. Protein Fitness Prediction	High (≥6 wks)	Medium	Medium–High (bio design)	High (Nemotron GPUs)
3. Drug Repurposing (L1000)	Medium (2–3 wks)	Medium	Medium (multi-disease)	Medium (A100×1–2)
4. Histopathology Classification	Medium (3–4 wks)	Low–Med	Medium (clinical AI)	Low–Medium (A100×1)
5. GPU scRNA Analysis (ScRAPIDS)	Low–Med (2–4 wks)	Low	Medium (tool dev)	Low (Tesla T4 or A100)
6. Biomedical Text Agent	High (6+ wks)	High (AI agent)	High (researcher aid)	High (Triton + GPUs)

- **Effort**: reflects development time for a polished solution (including refinement). Ideas 1,2,6 are multi-week; 3–5 could be done in a 72h sprint with simpler scope.
- **Novelty**: Federated projects and biomedical LLMs (ideas 1,6) are on the cutting edge. Histology/classification is well-trodden (lower novelty).
- **Impact**: Clinical applications (1,2,6) score high. Tools for data analysis (5) or repurposing (3) are also impactful.
- **GPU Needs**: Federated training and LLMs need many GPUs. Imaging/single-cell can use fewer GPUs or even GPUs on local machines.

Sources: The above ideas are inspired by hackathon projects【19†L329-L338】【20†L471-L480】 and research trends【40†L55-L60】【54†L35-L38】. All external data, tools, and architectures mentioned have supporting publications or repositories as cited.
